Official SDK & Developer Documentation

DeckX Control Studio — Plugin Developer Guide

Platform: DeckX Control Studio (.NET 8 / ESP32 Hardware) **Version:** 1.0.0 **Author:** Nactro Development Team

1. Introduction

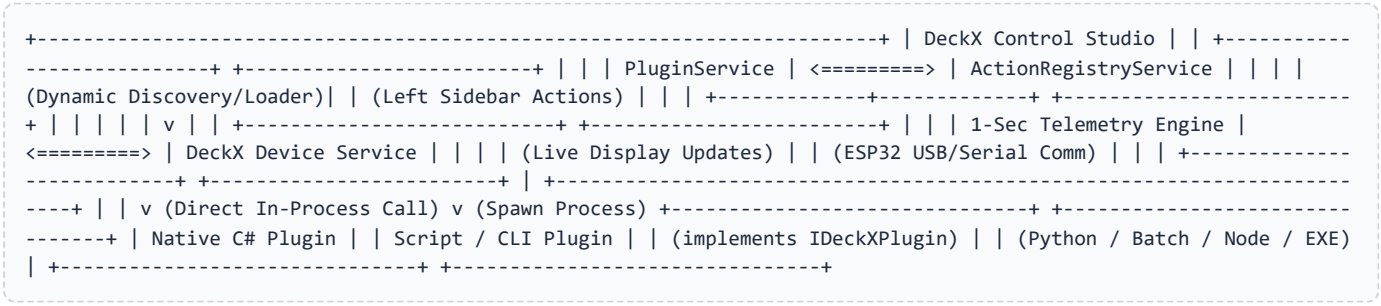
DeckX Control Studio supports an extensible, modular plugin architecture that enables developers to expand the 8-key Control Pad interface with:

- Third-party software automations (OBS Studio, Spotify, Discord, Home Assistant, Voicemeeter)
- Custom REST API integrations & webhooks
- Real-time hardware telemetry tickers (CPU %, RAM, Network speeds, Weather, Crypto/Stock tickers)
- Custom action macros and system utilities

Developers have two flexible paths for building plugins:

- **Option A: Script / Executable Plugins (Python, Node.js, Batch, PowerShell, C++, Go, Rust)**
  Zero C# knowledge required. Define your actions in a `plugin.json` manifest and execute any script or executable on key press.
- **Option B: Native .NET 8 / C# Plugins (IDeckXPlugin)**
  Compiled .NET class libraries with direct host access, supporting **1-second live telemetry tickers** (dynamically rewriting the button title, icon, and colors on the physical DeckX LCD screen in real time).

2. System Architecture



3. Option A: Creating Script Plugins (Python, Batch, etc.)

Script plugins are self-contained folders containing a `plugin.json` file and an entrypoint script or executable.

3.1 Plugin Folder Structure

```
my_media_plugin/
├── plugin.json               # Mandatory manifest
├── runner.bat / main.py      # Entrypoint script
└── assets/
    └── icon.png              # Action icon (optional)
```

3.2 Manifest Definition (`plugin.json`)

```
{
  "id": "com.nactro.mediacontrol",
  "name": "Media Controller",
  "author": "Nactro Dev",
  "version": "1.0.0",
  "description": "Control local media players via custom scripts",
  "category": "Media Control",
  "categoryIconGeometry": "M12 3v9.28a4.39 4.39 0 0 0 0-1.5-.28C8.01 12 6 14.01 6 16.5S8.01 21 10.5 21c2.31 0
  "executable": "runner.bat",
  "actions": [
    {
      "id": "action.play_pause",
      "title": "Play / Pause",
      "description": "Toggles active media playback",
      "iconGeometry": "M8 5v14l11-7z",
      "arguments": "--command play-pause"
    },
    {
      "id": "action.next_track",
      "title": "Next Track",
      "description": "Skips to next track",
      "iconGeometry": "M6 18l8.5-6L6 6v12zM16 6v12h2V6h-2z",
      "arguments": "--command next"
    },
    {
      "id": "action.prev_track",
      "title": "Previous Track",
      "description": "Returns to previous track",
      "iconGeometry": "M6 6h2v12H6zm3.5 6l8.5 6V6z",
      "arguments": "--command prev"
    }
  ]
}
```

3.3 Script Implementation Example

When a button is tapped, DeckX executes `executable` with the action's `arguments`.

runner.bat:

```
@echo off
python main.py %*
```

main.py:

```
import sys

if len(sys.argv) >= 3 and sys.argv[1] == "--command":
    cmd = sys.argv[2]
    if cmd == "play-pause":
        # Execute play/pause automation
        print("Toggling Play/Pause")
    elif cmd == "next":
        print("Next Track")
    elif cmd == "prev":
        print("Previous Track")
```

4. Option B: Native C# Plugins (High-Performance & Live Display)

Native plugins allow sub-millisecond execution and register **live key tickers** that dynamically rewrite button titles and icons every second on the hardware DeckX screen.

4.1 Class Library Project File (`.csproj`) & NuGet Setup

Install the official `DeckX.Core` NuGet package via CLI:

```
dotnet add package DeckX.Core --version 1.0.0
```

Configure your `.csproj` to reference the package with `PrivateAssets="all"` and `ExcludeAssets="runtime"` :

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <CopyLocalLockFileAssemblies>true</CopyLocalLockFileAssemblies>
  </PropertyGroup>

  <!-- Reference DeckX.Core from NuGet for standalone developer builds -->
  <ItemGroup Condition="!Exists('..\..\DeckX.Core\DeckX.Core.csproj')">
    <PackageReference Include="DeckX.Core" Version="1.0.0">
      <PrivateAssets>all</PrivateAssets>
      <ExcludeAssets>runtime</ExcludeAssets>
    </PackageReference>
  </ItemGroup>
</Project>
```

4.2 Implementing `IDeckXPlugin`

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using DeckX.Core.Models;
using DeckX.Core.Plugins;

namespace MyCustomPlugin;

public class SystemStatsPlugin : IDeckXPlugin
{
    public string Id => "com.developer.systemstats";
    public string Name => "System Stats Plugin";
    public string Author => "Developer Name";
    public string Version => "1.0.0";
    public string Description => "Real-time system telemetry plugin.";
    public string Category => "System Stats";
    public string CategoryIconGeometry => "M3 3h18v18H3V3zm16 16V5H5v14h14z";

    private IDeckXPluginContext? _context;
    private int _sampleCount = 0;

    public void OnInitialize(IDeckXPluginContext context)
    {
        _context = context;

        // Register 1-second live telemetry ticker
        context.RegisterLiveKey("action.live_counter", ctx =>
        {
            _sampleCount++;
            // Dynamically updates the button text on the physical LCD screen!
            ctx.SetTitle($"Count: {_sampleCount}");
        });
    }

    public IEnumerable<ActionItemModel> GetActions()
    {
        return new List<ActionItemModel>
        {
            new()
            {
                Id = "action.live_counter",
                Title = "Live Counter",
                Category = Category,
                ActionType = "Plugin: Live Counter",
                IconGeometry = "M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10 10-4.48 10-10S17.52 2 12 2zm0 18c-4
                Description = "Live hardware counter updated every second.",
                PluginId = Id
            }
        };
    }

    public Task OnActionTriggeredAsync(string actionId, DeckKeyActionContext context)
    {
        if (actionId == "action.live_counter")
        {
            _sampleCount = 0;
            context.SetTitle("Count: 0");
```

```
            context.ShowAlert("Counter has been reset!");
        }
        return Task.CompletedTask;
    }

    public void OnShutdown()
    {
        // Cleanup resources when DeckX studio exits
    }
}
```

5. `DeckKeyActionContext` & `IDeckXPluginContext` API Reference

5.1 `DeckKeyActionContext` (Key State & Visual Updates)

Passed to plugins during key presses, click events, periodic tickers, or push updates:

Method / Property	Type	Description
`ctx.SetTitle(string newTitle)`	Method	Immediately updates the key's label on both the PC Studio and the physical DeckX LCD screen.
`ctx.SetTitleColor(string hexColor)`	Method	Updates the label font color dynamically (e.g. `#00FF66` bright green, `#FF4444` red, `#FFAA00` amber).
`ctx.SetBackgroundColor(string hexColor)`	Method	Sets the button tile background color (e.g. `#0E2B18` dark green, `#331014` dark red, `#181B38` blurple).
`ctx.SetIcon(string? geom, string? path)`	Method	Swaps the icon SVG vector geometry or custom PNG bitmap file dynamically.
`ctx.ShowAlert(string message)`	Method	Triggers a visual confirmation toast in DeckX Control Studio.
`ctx.KeyIndex`	Property	Zero-based index (0-7) of the physical key currently assigned.
`ctx.CurrentTitle`	Property	The current text displayed on the key.

5.2 `IDeckXPluginContext` (Plugin Host Context)

Passed to plugins during `OnInitialize(IDeckXPluginContext context)`:

Method / Property	Type	Description
`context.RegisterLiveKey(actionId, callback)`	Method	Registers a 1-second periodic ticker that automatically executes and updates the key state.
`context.UnregisterLiveKey(actionId)`	Method	Removes a periodic ticker.
`context.TriggerKeyUpdate(actionId, updateAction)`	Method	**Instant Push Event:** Instantly updates assigned keys when an asynchronous background event occurs (IPC, WebSocket, serial) without waiting for the 1-second ticker!
`context.Log(string message)`	Method	Writes diagnostic logs to the DeckX debug stream.
`context.PluginDirectory`	Property	Root directory path where the plugin binaries and assets reside.
`context.DataDirectory`	Property	Persistent writable storage path for your plugin located at `%APPDATA%\DeckX\Plugins\<id>\data`.

6. Advanced Architecture: Background Two-Way Sync (Discord IPC Pattern)

For desktop applications (e.g., Discord, OBS Studio, Spotify) that run in the background or minimized, avoid relying solely on simulated keystrokes. Instead, implement a **Background IPC Client** (Named Pipes, WebSockets, or Local REST API):

1. **Connect to Local IPC:** In `OnInitialize`, start a background connection (e.g. Windows Named Pipe `\\.\pipe\discord-ipc-0`).
2. **Listen for Events:** Subscribe to app state changes (e.g. microphone mute/unmute).
3. **Instant Push Updates:** When the app state changes in the background, invoke `context.TriggerKeyUpdate("action_id", ctx => { ... })`.
4. **Hardware & UI Sync:** DeckX immediately repaints the physical LCD button with the updated title, SVG icon, and background color (e.g. Red for Muted, Green for Voice On).
5. **Background Command Execution:** When the user taps the DeckX key, dispatch the command directly over the IPC channel—working **100% reliably even when the target app is minimized or while playing a fullscreen game!**

7. Manifest Specification (`plugin.json`)

Field	Required	Type	Description
`id`	**Yes**	string	Unique reverse-DNS package identifier (e.g. `com.myname.spotify`).
`name`	**Yes**	string	Display name shown in Plugin Manager and Store.
`author`	**Yes**	string	Developer or company name.
`version`	**Yes**	string	Semantic versioning (e.g. `1.0.0`).
`category`	**Yes**	string	Section heading in the Left Sidebar Actions list.
`categoryIconGeometry`	No	string	SVG path data (M...z) for the category icon.
`executable`	*Cond.*	string	Relative path to script or exe (for Script plugins).
`assembly`	*Cond.*	string	Relative path to compiled .NET DLL (for C# plugins).
`actions`	**Yes**	array	List of action objects defined by the plugin.
`defaultPage`	No	object	**Auto-Setup Profile:** Optional default Deck page containing a wallpaper path and pre-configured keys. When installed, DeckX creates a dedicated themed page with this wallpaper and buttons automatically!

8. Packaging into `.deckxplugin`

To distribute your plugin to other users:

1. Place `plugin.json`, all scripts/binaries, and an `assets/` folder inside a single directory.
2. Select all items inside the directory and create a standard **ZIP archive**.
3. Rename the file extension from `.zip` to `.deckxplugin` (e.g. `MyPlugin.deckxplugin`).

> **Important:** `plugin.json` must be directly in the root of the ZIP archive, not inside a nested parent folder.

9. Installing & Importing Plugins

Method 1: Drag & Drop or UI Import (For End-Users)

1. Launch **DeckX Control Studio**.
2. Switch to the **Plugins** tab in the Left Sidebar or click **Install Plugin**.
3. Drag and drop the `.deckxplugin` file directly onto DeckX Studio.
4. DeckX unpacks the plugin to `%APPDATA%\DeckX\Plugins\<id>`, registers all actions, and immediately displays them in the sidebar without requiring an app restart!

Method 2: Manual Development Folder (For Developers)

1. Press `Win + R`, type `%APPDATA%\DeckX\Plugins`, and hit Enter.
2. Create a folder with your plugin's ID (e.g. `com.developer.myplugin`).
3. Place your `plugin.json`, scripts, and assets directly inside this directory.
4. Open or restart DeckX Control Studio. DeckX automatically scans and registers all subfolders on startup!

10. Best Practices & Troubleshooting

- **Relative Paths:** Always use relative paths from the plugin directory when launching external scripts or loading assets.
- **JSON Syntax:** Ensure `plugin.json` does not contain trailing commas or syntax errors.
- **Data Isolation:** Never write user data directly to the plugin root; use `context.DataDirectory` so user preferences survive plugin updates.
- **Clean Uninstallation:** When a plugin is deleted via the DeckX Studio UI, DeckX cleanly calls `OnShutdown()` and removes the registered actions and folder.

> **Ready for Distribution:** Developers can share their `.deckxplugin` packages with any DeckX user worldwide!